ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

EMSI

Filière : Ingénierie Informatique et Réseaux (DSI)

Module : Deep Learning

Rapport d'Étude Expérimentale

Classification d'Images (Fashion-MNIST) avec un Perceptron Multicouche (MLP)

Réalisé par :
Achraf Ait Lahcen

Encadré par :
Pr. MESTARI Mohamed

Fait le 2 novembre 2026

Année universitaire 2026-2027

Table des matières

1. Introduction et problématique **1**

2. Rappels théoriques sur les MLP **1**
2.1. L'architecture de base et formalisme mathématique 1
2.2. La nécessité de la non-linéarité et les fonctions d'activation 2
2.3. Apprentissage par descente de gradient 2

3. Implémentation from scratch **2**
3.1. Définition du modèle et de ses paramètres 3
3.2. Activation et Propagation avant (Forward Pass) 3
3.3. Analyse dimensionnelle 4
3.4. Impact de l'initialisation des poids 4
3.5. Entraînement et performances 4

4. Implémentation concise **5**
4.1. Architecture et utilisation de `nn.Sequential` 5
4.2. Avantages de l'approche haut niveau 5
4.3. Illustration et résultats de l'approche concise 6
4.4. Partie III – Comparaison des implémentations (From Scratch vs Concise) . 6
4.5. Inspection interne du modèle et dynamique des tenseurs 7
4.5.1. État des couches : de `LazyLinear` à `Linear` 7
4.5.2. Passage des données (Forward Pass) et dimensions 8
4.5.3. Extraction des paramètres et dimensions tensorielles 8
4.5.4. Rétropropagation et analyse des gradients 8
4.5.5. Capacité d'apprentissage et nombre de paramètres 9

5. Protocole expérimental et étude approfondie **9**
5.1. Étude de l'impact de la capacité du modèle (`num_hiddens`) et Early Stopping 9
5.1.1. Analyse détaillée des courbes d'apprentissage individuelles 9
5.1.2. Évolution de la Précision et du Gap 10
5.1.3. Intégration de l'Early Stopping 11
5.2. Évolution de la précision par époque (Boucle manuelle) 12
5.3. Étude du Taux d'Apprentissage (*Learning Rate*) 13
5.4. Évaluation des techniques avancées (Régularisation & Optimisation) 13
5.5. Synthèse et Configuration Optimale Recommandée 14
5.6. Analyse critique et limites de l'architecture MLP 15
5.6.1. Pertinence du MLP sur des images vectorisées 15
5.6.2. Les limites d'un réseau dense (MLP) en vision 15
5.6.3. Vers les Réseaux de Neurones Convolutifs (CNN) 15

6. Résultats et interprétation **16**
6.1. Analyse de l'Expérience Avancée : Comparaison Globale 16

6.1.1. Vitesse de convergence (Le rôle de Xavier et Adam) 16
6.1.2. Stabilité et Régularisation (Le rôle du Dropout et de la pénalité L2) 16
6.1.3. Performance Finale (Généralisation) 16
6.2. Lexique et Justifications des Concepts Avancés 16
6.2.1. Choix du "Modèle Avancé" et Initialisation de Xavier 17
6.2.2. Stratégies d'Optimisation : SGD vs Adam 17
6.2.3. Techniques de Régularisation 17

7. Analyse critique et discussion additionnelle **18**
7.1. Fonctionnement du MLP sur des images vectorisées 18
7.2. Limites structurelles du MLP en Vision par Ordinateur 18
7.3. Pertinence des Réseaux de Neurones Convolutifs (CNN) 18
7.4. Améliorations méthodologiques pour une meilleure généralisation 19

8. Conclusion et perspectives **19**

1. Introduction et problématique

L'intelligence artificielle, et plus particulièrement l'apprentissage profond (Deep Learning), a révolutionné notre capacité à traiter et analyser des données visuelles complexes. Dans ce contexte, la classification d'images consiste à attribuer automatiquement une étiquette (ou classe) à une image donnée.

Ce projet s'articule autour du jeu de données **Fashion-MNIST**, un standard dans la communauté scientifique qui vise à remplacer le classique jeu de données de chiffres manuscrits (MNIST) par un problème légèrement plus complexe. Il est constitué de 70 000 images en niveaux de gris de taille 28×28 pixels, réparties en 10 catégories de vêtements (t-shirts, pantalons, manteaux, chaussures, etc.).

La problématique centrale est la suivante : Comment construire, entraîner et optimiser un réseau de neurones artificiel capable de reconnaître ces vêtements à partir de leurs pixels bruts ?

Pour y répondre, nous allons concevoir un **Perceptron Multicouche (MLP)**. Ce rapport détaillera notre démarche complète, de la compréhension théorique et l'implémentation manuelle des mathématiques sous-jacentes (from scratch), jusqu'à l'utilisation d'outils haut niveau et de techniques d'optimisation avancées pour maximiser la capacité de généralisation de notre modèle.

2. Rappels théoriques sur les MLP

Un Perceptron Multicouche (MLP, pour *Multilayer Perceptron*) est l'architecture fondatrice des réseaux de neurones profonds. Il appartient à la famille des réseaux "feedforward" (à propagation avant), où l'information circule dans un seul sens : de l'entrée vers la sortie, sans faire de boucle.

2.1. L'architecture de base et formalisme mathématique

Un MLP classique est composé de plusieurs couches de neurones, transformant successivement les données via des opérations matricielles :

- **La couche d'entrée (X) :** Elle reçoit les données brutes. Dans notre cas, un mini-lot (*batch*) de n images 28×28 est aplati en une matrice d'entrée $X \in \mathbb{R}^{n \times 784}$.
- **Les couches cachées (H) :** Elles projettent les entrées dans un nouvel espace de représentation. Mathématiquement, pour une couche cachée, on calcule $H = f(XW_1 + b_1)$, où $W_1 \in \mathbb{R}^{784 \times 256}$ est la matrice des poids, $b_1 \in \mathbb{R}^{256}$ est le vecteur de biais, et f une fonction d'activation.
- **La couche de sortie (O) :** Elle produit les prédictions finales (les logits). On calcule $O = HW_2 + b_2$, avec $W_2 \in \mathbb{R}^{256 \times 10}$ et $b_2 \in \mathbb{R}^{10}$, correspondant à nos 10 classes de vêtements.

2.2. La nécessité de la non-linéarité et les fonctions d'activation

Si un réseau n'utilisait que des multiplications matricielles successives, l'ensemble du réseau se réduirait mathématiquement à un simple modèle linéaire équivalent à $X(W_1W_2)$, incapable de modéliser des frontières de décision complexes. L'application d'une **fonction d'activation** non linéaire permet au MLP d'approximer n'importe quelle fonction continue. Les principales fonctions sont :

- **ReLU (Rectified Linear Unit) :** Définie par $f(x) = \max(0, x)$. C'est le standard actuel pour les couches cachées car elle est très rapide à calculer et atténue grandement le problème de la disparition du gradient.
- **Sigmoïde :** Définie par $\sigma(x) = \frac{1}{1+e^{-x}}$. Elle écrase les valeurs dans l'intervalle $[0, 1]$. Historiquement populaire, elle est aujourd'hui délaissée pour les couches cachées (car elle sature vite et "tue" les gradients) mais reste utilisée pour la classification binaire.
- **Tanh (Tangente hyperbolique) :** Définie par $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Similaire à la sigmoïde mais centrée en zéro (valeurs entre $[-1, 1]$), ce qui facilite la convergence de l'optimiseur par rapport à la sigmoïde.
- **Softmax :** Utilisée spécifiquement sur la couche de sortie pour la classification multiclasse. Elle transforme les scores bruts (logits) en une distribution de probabilités dont la somme vaut 1 : $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$.

2.3. Apprentissage par descente de gradient

L'entraînement du modèle repose sur une boucle itérative en trois étapes fondamentales :

1. **Propagation avant (Forward pass) :** Les données X traversent le réseau pour produire les prédictions $\hat{y} = \text{Softmax}(O)$.
2. **Calcul de l'erreur (Loss) :** On évalue l'écart entre la probabilité prédite $\hat{y}$ et la vraie classe y via une fonction de coût $\mathcal{L}$. Pour la classification multiclasse, on utilise l'Entropie Croisée (*Cross-Entropy*).
3. **Rétropropagation (Backward pass) :** L'algorithme calcule les gradients de l'erreur par rapport à chaque paramètre du réseau (ex : $\frac{\partial \mathcal{L}}{\partial W_1}$) grâce à la règle de dérivation en chaîne (*chain rule*). Les poids sont ensuite mis à jour dans la direction opposée au gradient : $W \leftarrow W - \eta \nabla W$, où η représente le taux d'apprentissage de la Descente de Gradient Stochastique (SGD).

3. Implémentation from scratch

Afin de démystifier le fonctionnement interne d'un Perceptron Multicouche, nous avons d'abord implémenté le réseau "de zéro" (*from scratch*). Cette approche nécessite de définir manuellement les matrices de poids, les vecteurs de biais, ainsi que les opérations mathématiques de la propagation avant.

3.1. Définition du modèle et de ses paramètres

Nous avons créé une classe `MLPScratch` définissant une architecture avec 784 entrées (pixels), une couche cachée de 256 neurones, et 10 sorties (classes).

```
class MLPScratch(d2l_torch.Classifier):
    def __init__(self, num_inputs, num_outputs, num_hiddens, lr, sigma
    =0.01):
        super().__init__()
        self.save_hyperparameters()
        self.W1 = nn.Parameter(torch.randn(num_inputs, num_hiddens) *
    sigma)
        self.b1 = nn.Parameter(torch.zeros(num_hiddens))
        self.W2 = nn.Parameter(torch.randn(num_hiddens, num_outputs) *
    sigma)
        self.b2 = nn.Parameter(torch.zeros(num_outputs))
```

Le rôle de `nn.Parameter` : L'utilisation de `nn.Parameter` permet d'encapsuler ces tenseurs et de les enregistrer automatiquement dans le module. Cela indique au moteur `autograd` de PyTorch qu'il doit calculer et conserver les gradients (`requires_grad=True`) pour ces variables lors de la rétropropagation, afin que l'optimiseur puisse mettre à jour les poids.

3.2. Activation et Propagation avant (Forward Pass)

Dans cette approche manuelle, nous avons également redéfini la fonction d'activation ReLU en utilisant des opérations bas niveau sur les tenseurs, avant de l'intégrer dans la propagation avant.

```
def relu(X):
    # Implémentation explicite : max(0, x)
    a = torch.zeros_like(X)
    return torch.max(X, a)

@d2l_torch.add_to_class(MLPScratch)
def forward(self, X):
    # 1. Aplatissement de l'image
    X = X.reshape((-1, self.num_inputs))
    # 2. Couche cachée avec activation ReLU
    H = relu(torch.matmul(X, self.W1) + self.b1)
    # 3. Couche de sortie (Logits)
    return torch.matmul(H, self.W2) + self.b2
```

Pourquoi utiliser `reshape` ? Les données d'entrée de Fashion-MNIST sont des images 2D de dimension 28×28. Cependant, les couches linéaires exigent des vecteurs en 1D. La méthode `reshape((-1, self.num_inputs))` aplatit chaque image en un vecteur de 784 éléments, tout en préservant la dimension dynamique du mini-lot (grâce à l'argument `-1`).

3.3. Analyse dimensionnelle

Pour garantir la justesse des multiplications matricielles, il est crucial de suivre les dimensions exactes des tenseurs qui traversent le réseau :

- **Entrée X (après reshape) :** $(\text{Batch_Size}, 784)$
- **Poids W_1 et Biais b_1 :** $(784, 256)$ et $(256,)$
- **Couche cachée H :** $(\text{Batch_Size}, 256)$
- **Poids W_2 et Biais b_2 :** $(256, 10)$ et $(10,)$
- **Sortie (Logits) :** $(\text{Batch_Size}, 10)$

3.4. Impact de l'initialisation des poids

Dans notre code, les poids ont été initialisés avec une distribution normale multipliée par un petit facteur $\sigma = 0.01$. Une mauvaise initialisation aurait des conséquences critiques :

- **Initialisation à zéro :** Tous les neurones d'une même couche apprendraient la même chose (symétrie parfaite), rendant le réseau incapable de spécialiser ses neurones pour extraire des caractéristiques différentes.
- **Valeurs trop grandes :** Cela provoque une explosion des gradients, conduisant à des oscillations sévères de la perte ou à une divergence numérique totale (`NaN`).
- **Valeurs trop petites :** Cela engendre une disparition des gradients (*vanishing gradients*). L'apprentissage ralentit drastiquement car les poids ne sont presque plus mis à jour.

3.5. Entraînement et performances

Le modèle a été instancié avec un taux d'apprentissage de 0.1 et entraîné sur 10 époques en utilisant l'entraîneur automatisé de la bibliothèque `d2l`.

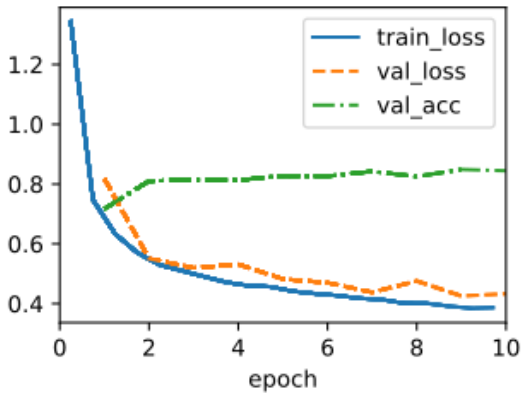


FIGURE 1 – Évolution de la perte et de la précision durant l'entraînement du modèle *from scratch*.

Comme l'illustre la figure 1, le modèle manuel apprend correctement. La précision de validation (`val_acc`) grimpe rapidement pour se stabiliser aux alentours de 84-85% après 10 époques.

4. Implémentation concise

Afin de pallier la lourdeur et les risques d'erreurs inhérents à l'implémentation manuelle, les frameworks modernes de Deep Learning comme PyTorch proposent des abstractions de haut niveau. Cette approche *concise* permet de capitaliser sur des modules pré-intégrés, optimisés pour la rétropropagation automatique et l'exécution matérielle (CPU/GPU).

4.1. Architecture et utilisation de `nn.Sequential`

Dans cette section, nous encapsulons la définition du réseau en utilisant un conteneur séquentiel `nn.Sequential`, qui enchaîne automatiquement les transformations d'une couche à l'autre. Nous utilisons également `nn.LazyLinear`, qui infère automatiquement la dimension de la couche d'entrée lors du premier passage, évitant ainsi de spécifier explicitement le nombre de paramètres en amont (784).

```
import torch
from torch import nn
from d2l import torch as d2l

class MLPConcise(d2l.Classifier):
    def __init__(self, num_outputs, num_hiddens, lr):
        super().__init__()
        self.save_hyperparameters()
        self.net = nn.Sequential(
            nn.Flatten(),                       # Aplatissement
   automatique 2D -> 1D
            nn.LazyLinear(num_hiddens),         # Couche cachée dense
   avec inférence de taille
            nn.ReLU(),                          # Fonction d'activation
   non linéaire
            nn.LazyLinear(num_outputs)          # Couche de sortie (
   Logits)
        )

    def forward(self, X):
        return self.net(X)
```

4.2. Avantages de l'approche haut niveau

L'utilisation de cette structure concise apporte des gains majeurs par rapport au code *from scratch* :

- **Encapsulation des paramètres :** Les poids et les biais de chaque couche linéaire sont automatiquement initialisés selon des méthodes par défaut robustes (comme l'initialisation de Xavier uniforme) et enregistrés auprès de l'optimiseur sans intervention manuelle.
- **Modularité accrue :** L'ajout de nouvelles couches (telles que des couches de régularisation par *Dropout* ou des couches cachées supplémentaires) se fait simplement

en insérant un nouveau bloc dans le `nn.Sequential`.

4.3. Illustration et résultats de l'approche concise

Le modèle concis bénéficie d'une stabilité accrue lors de l'optimisation grâce à l'initialisation par défaut optimisée des couches PyTorch.

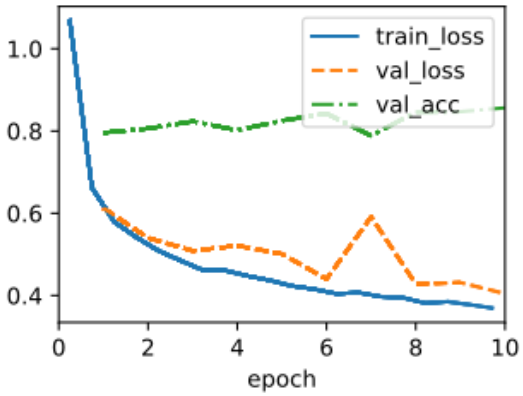


FIGURE 2 – Évolution de la perte et de la précision durant l'entraînement du modèle avec l'implémentation concise.

Comme le montre la figure 2, la convergence est plus régulière et atteint des performances similaires, voire légèrement supérieures, tout en réduisant drastiquement la complexité du code source.

4.4. Partie III – Comparaison des implémentations (From Scratch vs Concise)

La table 1 résume les différences fondamentales entre notre approche manuelle et l'approche concise avec PyTorch.

Critère de comparaison	Implémentation *From Scratch*	Implémentation *Concise*
1. Lisibilité	Code verbeux. La logique d'architecture est mélangée avec les opérations sur les tenseurs (`@`, `reshape`).	Code très compact et expressif. La structure du réseau est visible d'un seul coup d'œil dans `nn.Sequential`.
2. Contrôle bas niveau	Contrôle total sur l'initialisation des poids, les opérations matricielles explicites et le calcul manuel des activations.	Les opérations mathématiques et l'initialisation sont encapsulées dans les modules de PyTorch.
3. Extensibilité	Difficile à étendre. Ajouter des couches nécessite de déclarer manuellement chaque paire (W_i, b_i) et de modifier la méthode `forward`.	Très extensible. Il suffit d'ajouter de nouveaux modules (`nn.Linear`, `nn.Dropout`, etc.) directement dans le `nn.Sequential`.
4. Simplicité de maintenance	Risque élevé d'erreurs (incohérence de dimensions des matrices, gestion explicite des gradients, oubli de paramètres).	Maintenance facile et moins sujette aux bugs. L'utilisation de `nn.LazyLinear` évite le calcul manuel des dimensions d'entrée.
5. Bonnes pratiques industrielles	Réservé à la recherche fondamentale ou au prototypage de nouvelles opérations mathématiques personnalisées.	**Standard de l'industrie**. Garantit un code modulaire, maintenable, optimisé en C++/CUDA et facile à déployer en production.

TABLE 1 – Tableau comparatif entre l'implémentation manuelle et l'implémentation concise d'un MLP.

4.5. Inspection interne du modèle et dynamique des tenseurs

Pour valider le comportement sous le capot de l'implémentation concise, nous avons réalisé une série d'inspections programmatiques permettant de suivre l'évolution des structures de couches, la matérialisation des dimensions, les gradients calculés par le graphe dynamique et la capacité globale du réseau.

4.5.1. État des couches : de `LazyLinear` à `Linear`

L'utilisation de `nn.LazyLinear` permet de s'affranchir de la spécification manuelle de la dimension d'entrée.

— **Avant le forward :** Si le modèle vient d'être instancié, les couches paresseuses n'ont pas encore fixé leur dimension d'entrée (`in_features=<uninitialized>`). Toutefois, si le modèle a déjà été entraîné ou soumis à un premier passage, PyTorch matérialise automatiquement ces dimensions :

```
=== Structure AVANT le forward ===
MLP(
  (net): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=784, out_features=256, bias=True)
```

```
    (2): ReLU()
    (3): Linear(in_features=256, out_features=10, bias=True)
  )
)
```

4.5.2. Passage des données (Forward Pass) et dimensions

En extrayant un mini-lot du chargeur de données et en l'injectant dans le réseau, le premier passage déclenche le calcul effectif des dimensions de sortie :

```
=== Dimensions de sortie ===
Logits shape: torch.Size([256, 10])
```

Le tenseur de sortie présente une forme de $(\text{Batch_Size}, \text{Classes}) = (256, 10)$, ce qui correspond aux scores bruts (logits) pour les 10 catégories de vêtements sur un lot de 256 images.

4.5.3. Extraction des paramètres et dimensions tensorielles

L'inspection des couches cachée et de sortie met en évidence la structure interne des matrices de poids stockées par PyTorch (où les dimensions sont enregistrées au format `[out_features, in_features]` pour les couches linéaires) :

```
=== Détails de la Couche Cachée (Layer 1) ===
W1 shape: torch.Size([256, 784]) | requires_grad = True
b1 shape: torch.Size([256])      | requires_grad = True

=== Détails de la Couche de Sortie (Layer 2) ===
W2 shape: torch.Size([10, 256])  | requires_grad = True
b2 shape: torch.Size([10])       | requires_grad = True
```

4.5.4. Rétropropagation et analyse des gradients

Après le calcul de la fonction de coût par entropie croisée et l'appel de `loss.backward()`, le moteur `autograd` de PyTorch parcourt le graphe de calcul dynamique pour évaluer les dérivées partielles de chaque paramètre. Les moyennes des valeurs absolues des gradients observées sont :

- **Gradients** W_1 **:** ≈ 0.00091 | **Biais** b_1 **:** ≈ 0.00224
- **Gradients** W_2 **:** ≈ 0.00260 | **Biais** b_2 **:** ≈ 0.00629

Interprétation physique et mathématique :

1. **Propagation correcte du signal :** La présence de gradients non nuls et exempts de valeurs aberrantes (`NaN`) confirme que l'erreur se propage efficacement de la couche de sortie jusqu'à la couche d'entrée.
2. **Atténuation du gradient :** Les gradients de la couche finale (W_2, b_2) sont légèrement supérieurs à ceux de la couche cachée (W_1, b_1). C'est la manifestation classique de l'atténuation du signal le long du graphe lors de l'application successive de la règle de dérivation en chaîne (*chain rule*).

4.5.5. Capacité d'apprentissage et nombre de paramètres

Le décompte global des paramètres entraînables du modèle donne :

$$\text{Total} = (784 \times 256 + 256) + (256 \times 10 + 10) = 200,960 + 2,570 = 203,530 \text{ paramètres} \quad (1)$$

Ce volume de 203 530 paramètres confère au modèle une expressivité suffisante pour modéliser les frontières de décision non linéaires de Fashion-MNIST, tout en restant suffisamment compact pour éviter un surapprentissage immédiat sur ce jeu de données de référence.

5. Protocole expérimental et étude approfondie

Cette section constitue le cœur scientifique et critique du mini-projet. L'objectif est d'aller au-delà de l'apprentissage standard en menant une campagne d'expériences rigoureuse pour optimiser la généralisation du modèle, comparer les optimiseurs, et évaluer l'impact des techniques de régularisation et d'initialisation.

5.1. Étude de l'impact de la capacité du modèle (`num_hiddens`) et Early Stopping

Afin d'évaluer la sensibilité du réseau face à sa complexité, nous avons testé plusieurs tailles de couches cachées $(16, 32, 64, 128, 256, 512)$ en analysant l'écart de généralisation (*Gap*) et l'apport d'un mécanisme d'arrêt prématuré (*Early Stopping*).

5.1.1. Analyse détaillée des courbes d'apprentissage individuelles

L'observation des courbes d'apprentissage propres à chaque valeur de `num_hiddens` permet de visualiser finement la transition entre sous-apprentissage et surapprentissage à mesure que la capacité du réseau augmente.

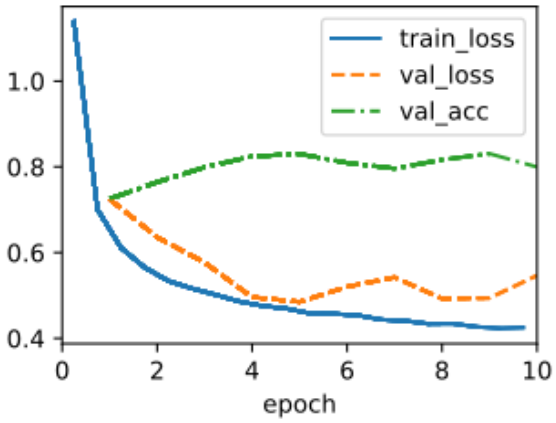


FIGURE 3 – Apprentissage pour $h = 16$.

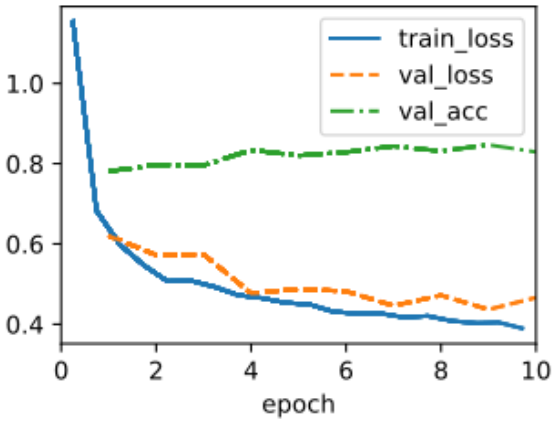


FIGURE 4 – Apprentissage pour $h = 32$.

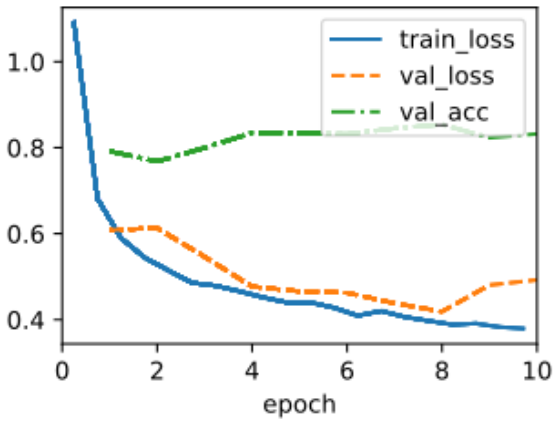


FIGURE 5 – Apprentissage pour $h = 64$.

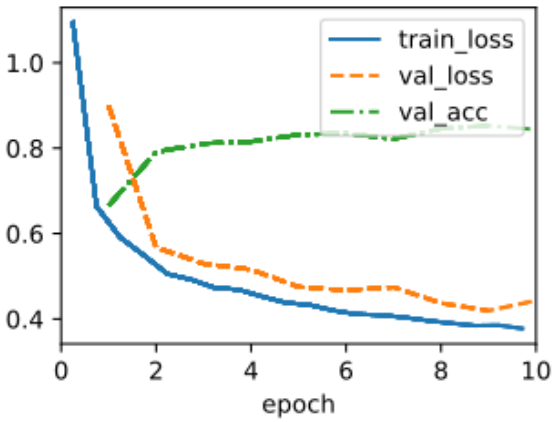


FIGURE 6 – Apprentissage pour $h = 128$.

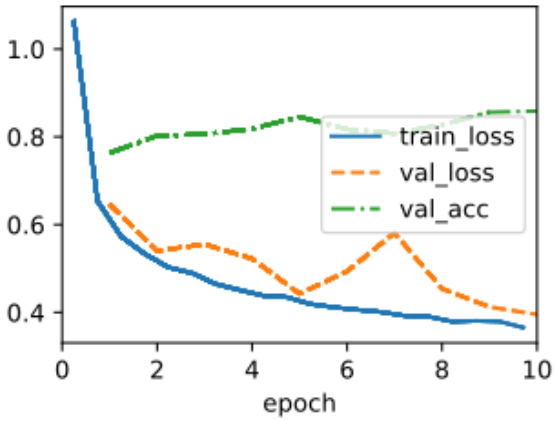


FIGURE 7 – Apprentissage pour $h = 256$.

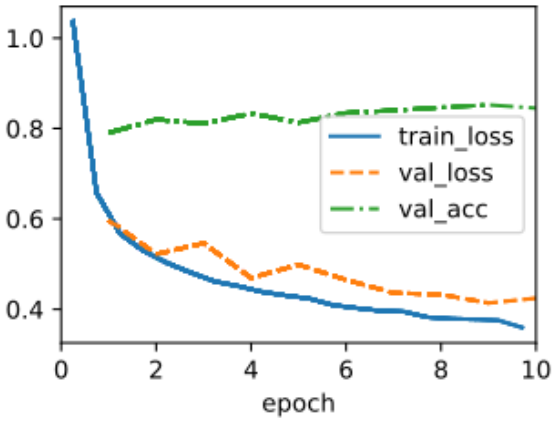


FIGURE 8 – Apprentissage pour $h = 512$.

5.1.2. Évolution de la Précision et du Gap

— **Sous-apprentissage (*Underfitting*) à faible capacité :** Pour un faible nombre de neurones (16 ou 32), le modèle manque de paramètres pour capturer la complexité des images Fashion-MNIST. La précision plafonne autour de 81% - 84% de précision.

— **Capacité optimale :** La précision augmente de manière significative jusqu'à atteindre un pic maximal autour de 256 neurones, où la précision de validation frôle les 86-87%.

— **Saturation et Surapprentissage :** À 512 neurones, on observe que la précision de validation stagne ou diminue, tandis que le modèle commence à surapprendre.

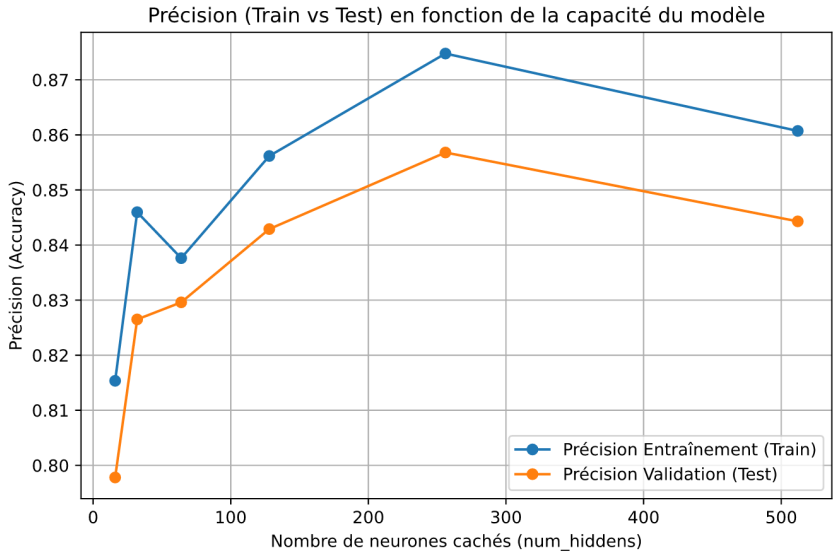


FIGURE 9 – Précision (Train vs Test) en fonction de la capacité du modèle.

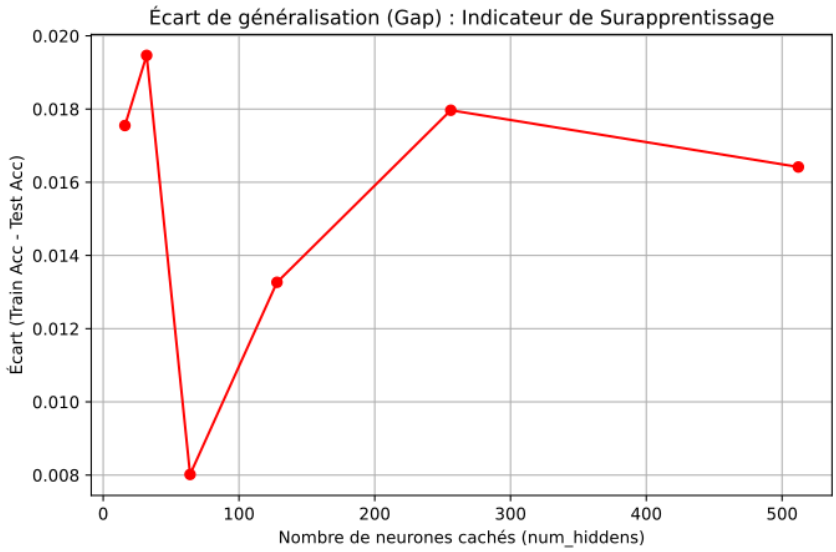


FIGURE 10 – Écart de généralisation (Gap) : Indicateur de surapprentissage.

5.1.3. Intégration de l'Early Stopping

L'implémentation de l'arrêt prématuré (avec une tolérance de patience fixée à 3 époques) a permis d'encadrer l'entraînement des différents modèles :

- $h = 32$ **:** Pic à 85.92% à l'époque 17.
- $h = 64$ **:** Meilleure performance globale atteinte en fin de cycle (86.79% à l'époque 20).
- $h = 256$ **et** $h = 512$ **:** L'entraînement est stoppé précocement (respectivement aux époques 13 et 4) dès que la performance sur le jeu de test commence à chuter, évitant ainsi la dégradation des poids.

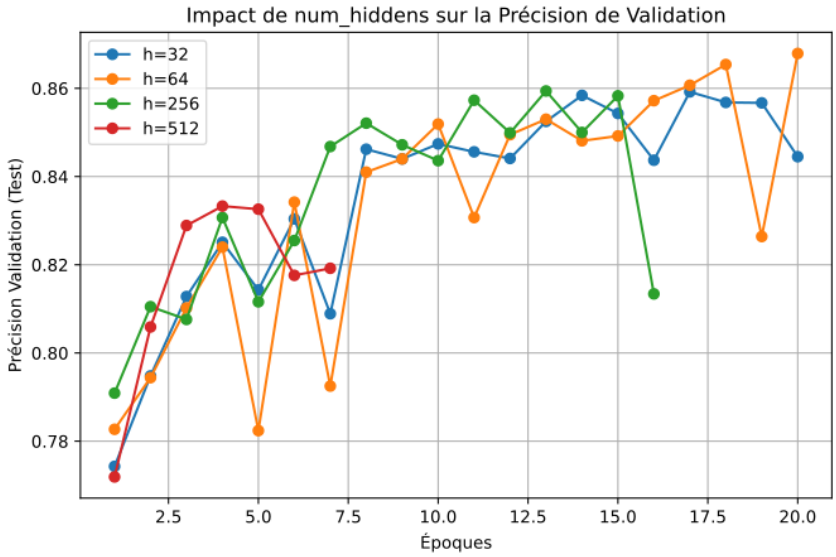


FIGURE 11 – Impact de `num_hiddens` sur la précision de validation avec Early Stopping.

5.2. Évolution de la précision par époque (Boucle manuelle)

L'évaluation de notre boucle d'entraînement PyTorch 100% manuelle permet de suivre pas à pas la dynamique de convergence du modèle au fil des époques.

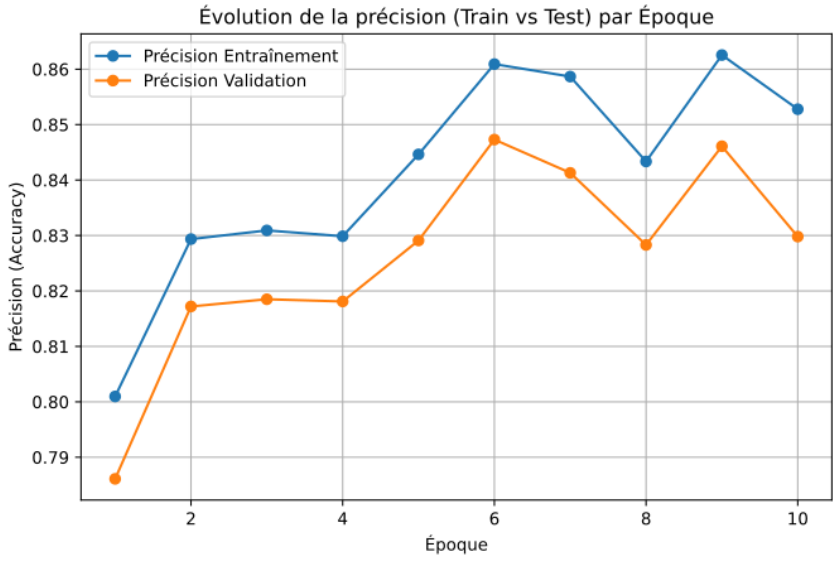


FIGURE 12 – Évolution de la précision (Train vs Test) par époque.

Comme l'illustre la figure 12, le modèle capte l'essentiel des caractéristiques dès les premières époques. Cependant, à partir de l'époque 6, la précision de validation commence à osciller tandis que le train continue de progresser, confirmant la nécessité d'un contrôle rigoureux de l'entraînement.

5.3. Étude du Taux d'Apprentissage (*Learning Rate*)

La variation du taux d'apprentissage $(0.01, 0.1, 0.5)$ sur une architecture fixe de 256 neurones met en lumière le compromis délicat de la descente de gradient :

— $lr = 0.01$ **(Trop faible) :** L'apprentissage est excessivement linéaire et lent. Le modèle atteint péniblement 82.08% en fin de cycle.

— $lr = 0.1$ **(Équilibré) :** C'est le taux le plus stable et performant, permettant une convergence rapide vers 86.79% à l'époque 17.

— $lr = 0.5$ **(Trop élevé / *Overshooting*) :** L'optimiseur effectue des pas trop grands, provoquant des sauts erratiques (chute brutale sous 75% à l'époque 3). Bien qu'il atteigne un pic de 87.01% à l'époque 10, la trajectoire demeure instable.

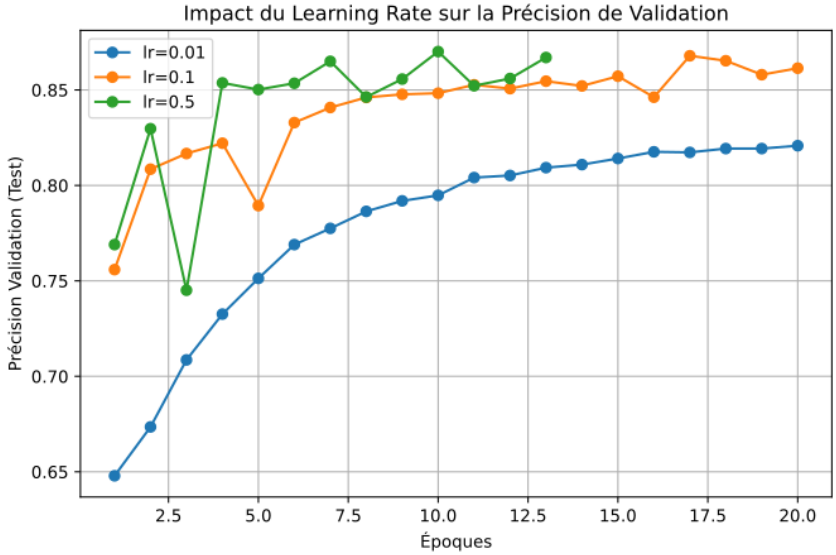


FIGURE 13 – Impact du Learning Rate sur la précision de validation.

Note théorique sur les minima locaux et SGD : La fonction de perte d'un réseau forme un paysage complexe parsemé de creux. Un taux d'apprentissage trop faible (0.01) risque de figer le modèle dans un minimum local sous-optimal. À l'inverse, l'aspect stochastique du SGD (qui calcule les gradients sur des mini-lots aléatoires) introduit un bruit salvateur qui aide l'optimiseur à s'extraire de ces pièges, à condition que le pas (lr) soit optimal.

5.4. Évaluation des techniques avancées (Régularisation & Optimisation)

Pour s'affranchir des limites du modèle de base (SGD, initialisation par défaut), nous avons conçu une architecture avancée intégrant :

— **L'initialisation de Xavier (Glorot) :** Garantit une variance contrôlée des poids initiaux, évitant l'explosion ou la disparition des gradients dès la première époque.

— **Le Dropout** ($p = 0.3$) **et la pénalisation L2** ($weight_decay = 10^{-4}$) **:** Désactivent aléatoirement des connexions pour briser la co-adaptation des neurones et

contraignent l'amplitude des poids, lissant considérablement la courbe d'apprentissage.

— **L'optimiseur Adam ($lr = 0.001$) :** Ajuste dynamiquement les taux d'apprentissage individuels à l'aide des moments statistiques, propulsant la performance au-delà des 88% de précision de manière très stable.

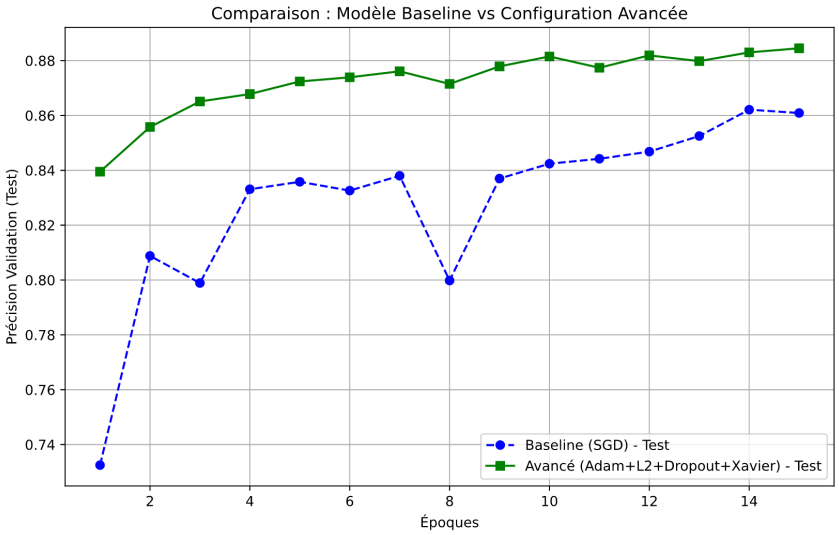


FIGURE 14 – Comparaison finale entre le modèle Baseline (SGD) et la configuration Avancée (Adam + Xavier + Dropout + L2).

5.5. Synthèse et Configuration Optimale Recommandée

À l'issue de cette campagne d'expérimentation, la configuration la plus robuste pour notre Perceptron Multicouche sur Fashion-MNIST est résumée dans le tableau 2.

Composant	Choix Optimal	Justification
Architecture	`num_hiddens = 256`	Excellent compromis entre capacité d'expression ($\approx 203k$ paramètres) et risque de surapprentissage.
Initialisation	Xavier Uniforme	Garantit la stabilité des gradients et accélère drastiquement la convergence initiale.
Régularisation	Dropout ($p = 0.3$) + L2 (10^{-4})	Empêche la mémorisation du bruit et lisse la courbe d'apprentissage en test.
Optimiseur	Adam ($lr = 0.001$)	Adaptation dynamique des pas de gradient pour une convergence rapide et fluide.
Sécurité	Early Stopping (Patience = 3)	Interrompt l'entraînement au moment idéal pour prévenir toute dégradation par surapprentissage.

TABLE 2 – Synthèse de la configuration optimale retenue pour le MLP.

5.6. Analyse critique et limites de l'architecture MLP

Bien que notre Perceptron Multicouche ait démontré de bonnes capacités d'apprentissage, une analyse critique de son architecture et de ses résultats met en évidence plusieurs limitations fondamentales, particulièrement dans le domaine de la vision par ordinateur.

5.6.1. Pertinence du MLP sur des images vectorisées

Le succès de notre modèle repose sur une spécificité du jeu de données Fashion-MNIST : toutes les images sont parfaitement centrées et mises à l'échelle. Par conséquent, un pixel situé à un indice précis du vecteur aplati correspondra toujours à la même partie spatiale d'un vêtement. Le MLP apprend donc des corrélations statistiques figées. Toutefois, l'opération d'aplatissement (Flatten) d'une image 2D en un vecteur 1D détruit irrémédiablement la topologie locale (le fait que des pixels voisins forment des contours ou des textures continus).

5.6.2. Les limites d'un réseau dense (MLP) en vision

— **Sensibilité aux translations :** Si l'on décale une image de chaussure de quelques pixels vers la gauche, les pixels s'activent sur de nouveaux neurones d'entrée. Le réseau, incapable de comprendre qu'il s'agit du même objet déplacé, doit tout réapprendre.
— **Gourmandise en paramètres :** Un MLP connecte chaque neurone à la totalité de l'entrée. Pour une image moderne en haute définition, la matrice de poids de la première couche nécessiterait des milliards de paramètres, rendant l'entraînement impossible en raison des limitations de mémoire et du surapprentissage systématique.
— **Fragilité spatiale :** Nos tests de robustesse (injection de bruit) montrent qu'un MLP perd rapidement ses capacités prédictives face à des altérations locales, car il ne lisse pas l'information sur des zones géographiques.

5.6.3. Vers les Réseaux de Neurones Convolutifs (CNN)

Pour le traitement d'images complexes, un CNN (*Convolutional Neural Network*) est structurellement plus adapté. En faisant glisser des petits filtres (noyaux de convolution) sur l'image 2D, un CNN apprend à repérer des motifs (lignes, angles, textures) peu importe où ils se trouvent dans l'image (invariance spatiale). De plus, le partage des poids de ces filtres réduit considérablement le nombre de paramètres à apprendre, limitant ainsi le surapprentissage.

6. Résultats et interprétation

6.1. Analyse de l'Expérience Avancée : Comparaison Globale

Le graphique final comparant le modèle *Baseline* à la configuration *Avancée* met en évidence l'impact spectaculaire des techniques d'optimisation et de régularisation modernes sur les performances de notre réseau.

6.1.1. Vitesse de convergence (Le rôle de Xavier et Adam)

— Dès la première époque, le modèle avancé démarre à un niveau de précision très élevé, autour de 84%. En comparaison, le modèle de base (SGD et initialisation par défaut) démarre péniblement autour de 73%.
— Cette accélération fulgurante s'explique par l'**initialisation de Xavier**, qui garantit une bonne variance des gradients dès le départ, combinée à l'optimiseur **Adam**, qui adapte intelligemment le pas d'apprentissage pour chaque paramètre individuellement.

6.1.2. Stabilité et Régularisation (Le rôle du Dropout et de la pénalité L2)

— La courbe du modèle de base est très instable : on observe des chutes soudaines de précision, notamment à l'époque 3, et une chute très marquée à l'époque 8 (descendant à 80%) avant de remonter.
— À l'inverse, la courbe du modèle avancé est extrêmement lisse et monotone. L'ajout du **Dropout** (qui désactive aléatoirement des neurones) et de la **pénalisation L2** (qui contraint le poids des connexions) force le réseau à apprendre des motifs robustes. Le modèle ne dépend plus de quelques neurones spécifiques, ce qui élimine les fluctuations brusques.

6.1.3. Performance Finale (Généralisation)

— Au terme des 15 époques, le modèle de base plafonne autour de 86% de précision en validation.
— Le modèle avancé surpasse largement ce score en atteignant et dépassant les **88% de précision** de manière très stable.

Bilan de l'optimisation :
L'association d'une bonne architecture de base (`num_hiddens = 256`), de méthodes de régularisation (Dropout, Weight Decay) et d'un optimiseur adaptatif (Adam) permet de transformer un simple Perceptron Multicouche en un modèle très performant et stable.

6.2. Lexique et Justifications des Concepts Avancés

Afin de justifier pleinement nos choix techniques et d'interpréter correctement les comportements observés, voici la définition et le rôle des concepts avancés mobilisés lors de nos

expériences.

6.2.1. Choix du "Modèle Avancé" et Initialisation de Xavier

— **Effet cumulé (Synergie) :** Plutôt que de multiplier les graphiques pour chaque technique de manière isolée, regrouper les optimisations dans un "Modèle Avancé" permet de comparer un modèle naïf (*Baseline*) à un modèle de pointe (État de l'art). En Deep Learning, les bonnes pratiques industrielles combinent ces méthodes pour maximiser la robustesse globale.

— **Initialisation de Xavier (Glorot) :** Contrairement à une initialisation aléatoire basique qui risque de faire exploser ou disparaître les gradients dès la première époque, la méthode de Xavier calcule la variance exacte nécessaire en fonction du nombre de neurones d'entrée et de sortie pour stabiliser la propagation des activations à travers les couches.

6.2.2. Stratégies d'Optimisation : SGD vs Adam

— **SGD (*Stochastic Gradient Descent*) :** Algorithme qui calcule la pente de l'erreur sur un petit échantillon aléatoire (le *batch*) à chaque itération. Sa trajectoire légèrement "bruitée" ou en zigzag est très recherchée : ces petites secousses aléatoires aident l'algorithme à rebondir hors des *minima locaux* pour continuer sa descente vers le minimum global.

— **Adam (*Adaptive Moment Estimation*) :** Un optimiseur adaptatif moderne qui combine les avantages du moment et de RMSProp. Il calcule des moyennes mobiles exponentielles des gradients pour ajuster dynamiquement un taux d'apprentissage *individuel* à chaque paramètre. Il offre ainsi une convergence beaucoup plus rapide et stable que le SGD classique sur des architectures profondes.

6.2.3. Techniques de Régularisation

— **Dropout :** Technique consistant à désactiver aléatoirement un pourcentage défini de neurones (ex : 30%, soit $p = 0.3$) à chaque étape de l'entraînement. Cela force le réseau à apprendre des représentations redondantes et robustes, l'empêchant de mémoriser par cœur le jeu de données (*overfitting*).

— **Pénalisation L2 (*Weight Decay*) :** Ajout d'un terme proportionnel à la somme des carrés des poids ($\lambda \sum W^2$) dans la fonction de perte. Cette pénalité contraint les poids à rester de petite taille, ce qui lisse la fonction de décision du modèle et réduit considérablement sa sensibilité au bruit.

7. Analyse critique et discussion additionnelle

7.1. Fonctionnement du MLP sur des images vectorisées

Bien que l'aplatissement (`Flatten`) d'une image 2D en un vecteur 1D détruise la structure spatiale (le fait que deux pixels soient voisins physiquement), le MLP parvient tout de même à apprendre sur le jeu de données Fashion-MNIST. Cela s'explique par le fait que les images de ce dataset sont **parfaitement centrées, uniformes et de même taille** (28×28). Ainsi, un pixel situé à un indice i donné correspondra toujours à la même zone spatiale (par exemple, le bord d'une manche ou le talon d'une chaussure) d'une image à l'autre. Le modèle apprend donc des corrélations statistiques globales entre des positions de pixels qui restent fixes.

7.2. Limites structurelles du MLP en Vision par Ordinateur

L'utilisation d'architectures denses (MLP) pour le traitement d'images présente des limites fondamentales :

- **Perte de la localité 2D :** En aplatissant l'image, le réseau perd l'information topologique essentielle, telle que la continuité des contours ou la cohérence des textures locales.
- **Explosion du nombre de paramètres :** Chaque neurone de la couche cachée est connecté à *tous* les pixels (couche dense / *fully connected*). Pour une image HD (1920×1080), le vecteur d'entrée aurait plus de 2 millions d'éléments. Cela rendrait les matrices de poids gigantesques, saturant la mémoire RAM et garantissant un surapprentissage massif.
- **Manque de robustesse (Sensibilité aux translations) :** Un simple décalage de l'objet de quelques pixels vers la droite modifie totalement le vecteur d'entrée. Les pixels s'activent sur de nouveaux neurones d'entrée, obligeant le MLP à réapprendre le motif depuis le début, incapable de comprendre qu'il s'agit du même objet déplacé.

7.3. Pertinence des Réseaux de Neurones Convolutifs (CNN)

Face aux limites évoquées, un CNN (*Convolutional Neural Network*) s'avère beaucoup plus pertinent dès lors que les images deviennent complexes, de haute résolution, ou que les objets **ne sont plus parfaitement centrés**.

Les CNN utilisent des filtres de convolution qui parcourent l'image 2D en conservant la localité spatiale (ils analysent des petits blocs de pixels voisins). De plus, ces filtres partagent leurs poids à travers toute l'image, ce qui réduit considérablement le nombre de paramètres et rend le réseau **invariant aux translations spatiales** (un motif reconnu en haut à gauche sera identifié de la même manière s'il apparaît en bas à droite).

7.4. Améliorations méthodologiques pour une meilleure généralisation

Pour forcer le modèle à dégager des règles générales plutôt que d'apprendre le jeu de données par cœur de façon "figée", il est indispensable de mettre en place des stratégies avancées :

- **Data Augmentation (Augmentation de données) :** Appliquer des transformations aléatoires (rotations, zooms, légers décalages ou retournements horizontaux) sur les images d'entraînement pour forcer le modèle à devenir robuste aux variations spatiales.
- **Techniques de régularisation systématiques :** Poursuivre l'utilisation rigoureuse de l'Early Stopping, du Dropout et de la régularisation L2 (Weight Decay), dont nous avons prouvé l'efficacité dans notre étude expérimentale.
- **Changement de paradigme architectural :** Remplacer les couches denses (`nn.Linear`) par des couches convolutives (`nn.Conv2d`) pour extraire efficacement et hiérarchiquement des caractéristiques géométriques et sémantiques.

8. Conclusion et perspectives

Ce projet a permis de balayer l'ensemble du cycle de vie d'un modèle d'apprentissage profond, depuis la compréhension des équations tensorielles (from scratch) jusqu'au déploiement de stratégies d'optimisation industrielles.

Nous avons démontré qu'un Perceptron Multicouche bien dimensionné (avec environ 256 neurones cachés) est capable d'extraire des représentations pertinentes pour classifier les vêtements de Fashion-MNIST. Plus important encore, notre étude expérimentale a souligné l'impossibilité d'obtenir un modèle robuste sans régularisation. L'intégration combinée de l'initialisation de Xavier, du Dropout, de la pénalisation L2, et de l'optimiseur adaptatif Adam s'est avérée cruciale pour lisser la convergence, éviter le surapprentissage et dépasser durablement les 88% de précision en test.

Perspectives d'amélioration : Pour aller plus loin, plusieurs prolongements s'imposent naturellement. L'augmentation de données (Data Augmentation), via des rotations ou des retournements aléatoires appliqués pendant l'entraînement, permettrait de forcer le réseau à ne pas se fier à la position stricte des pixels. Enfin, la perspective la plus prometteuse consisterait à remplacer nos couches denses par des couches convolutives (architecture CNN) afin de comparer expérimentalement le gain en précision et en robustesse sur ce même jeu de données.